Creación de Repositorio

DOCENTE:
Jorge Armando Julio Cruz

ESTUDIANTE:
Juan Sebastián Fandiño Urrego

CURSO:
Programación Orientada a Objetos II

Noviembre 2026

VIGILADO MINEDUCACIÓN

Índice

Introducción..... 3
Objetivos..... 3
Código fuente..... 3
- **Registro:..... 3**
- **GeneradorDatos:..... 4**
- **AnalizadorTransporte:..... 6**
- **Main:..... 10**
- **Conceptos funcionales**
  Funciones puras:..... 13
- **Inmutabilidad..... 13**
- **Expresiones lambda..... 13**

Introducción

Se mueven en la ciudad de TecnoValle a diario más de 450.000 personas, generando entre 6 y 12 millones de registros de transporte, con su plataforma actual, construida con un enfoque más imperativo, se puede tardar hasta dos horas en procesar los datos de un día; presenta entonces errores de conteo según las condiciones de carrera, adicionalmente, es costosa de mantener. Se crea como propuesta de mejora y solución a la misma, TecnoMovil Data, en busca de migrar a una arquitectura modular basada en programación funcional.

Objetivos

Objetivo general: Se busca implementar en Java un módulo de procesamiento de datos que aplique los principios de la programación funcional y procese de forma declarativa y sin efectos secundarios.
Objetivos específicos: Por medio de estructuras inmutables se busca modelar los registros de transporte, de manera que se genera un conjunto de datos de gran volumen y se resuelve mediante funciones puras los análisis requeridos.

Código fuente

Fue desarrollada la solución en Java 21, bajo el uso de la biblioteca estándar y el paquete com.tecnomovil, que cuenta con cuatro clases: **Registro**, **GeneradorDatos**, **AnalizadorTransporte** y **Main**.

Este se compila con ***javac -encoding UTF-8 -d out src\com\tecnomovil*.java*** y se ejecuta con ***java -cp out com.tecnomovil.Main***:

Registro:

```
package com.tecnomovil;

import java.time.LocalDateTime;
import java.util.Objects;


public record Registro(String idUsuario, String ruta, String estacion, Accion
accion, LocalDateTime timestamp) {

    public enum Accion {ENTRADA, SALIDA}
    public Registro {
        Objects.requireNonNull(idUsuario, "idUsuaio");
        Objects.requireNonNull(ruta, "ruta");
        Objects.requireNonNull(estacion, "estacion");
        Objects.requireNonNull(accion, "accion");
        Objects.requireNonNull(timestamp, "timestamp");
    }
}
```

GeneradorDatos:

```
package com.tecnomovil;

import com.tecnomovil.Registro.Accion;
import java.time.LocalDate;
import java.time.LocalDateTime;
import java.util.Comparator;
```

```java
import java.util.List;
import java.util.Map;
import java.util.Random;
import java.util.stream.IntStream;
import java.util.stream.Stream;

/*
Se genera conjunto de registros simulados.
Siendo determinista, la misma cantidad de usuarios y la misma semilla
producen siempre los mismos datos, devolviendo una lista inmutable.
*/
public final class GeneradorDatos {

    private GeneradorDatos() {  }
    public static final LocalDate FECHA = LocalDate.of(2026, 11, 5);

    /* Rutas y estaciones */
    public static final Map<String, List<String>> RUTAS = Map.of(
            "T1-Troncal Norte", List.of("Portal Norte", "Calle 100", "Calle
72", "Centro", "Museo"),
            "T2-Troncal Sur", List.of("Centro", "Ricaurte", "Calle 6",
"Portal Sur"),
            "T3-Av. Oriental", List.of("Museo", "Universidades", "Las Aguas",
"Calle 72"),
            "A1-Alim. Occidente", List.of("Portal Occidente", "Banderas",
"Ricaurte"),
            "A2-Alim. Oriente", List.of("Las Aguas", "La Paz", "San
Cristobal"));

    /* Rutas repetidas según demanda */
    private static final List<String> RUTAS_PONDERADAS = Stream.of(
                    Map.entry("T1-Troncal Norte", 4), Map.entry("T2-Troncal
Sur", 3),
                    Map.entry("T3-Av. Oriental", 2), Map.entry("A1-Alim.
Occidente", 2),
                    Map.entry("A2-Alim. Oriente", 1))
            .flatMap(e -> Stream.generate(e::getKey).limit(e.getValue()))
            .toList();

    /* Franjas horarias, con mayor probabilidad en horas pico */
    private static final List<Integer> MANANA = List.of(5, 6, 6, 7, 7, 7, 8,
8, 9);
    private static final List<Integer> MEDIODIA = List.of(11, 12, 12, 13,
14);
    private static final List<Integer> TARDE = List.of(16, 17, 17, 18, 18,
18, 19, 20);

    /* Genera todos los registros del día */
    public static List<Registro> generar(int cantidadUsuarios, long semilla)
{
        return IntStream.range(0, cantidadUsuarios)
                .parallel()
                .boxed()
                .flatMap(i -> viajesDeUsuario(i, semilla))

.sorted(Comparator.comparing(Registro::timestamp).thenComparing(Registro::idU
suario))
                .toList(); // lista inmutable
    }

    /* Viajes de un usuario, de 1 a 3 viajes */
    private static Stream<Registro> viajesDeUsuario(int indice, long semilla)
{
```

```
        Random rnd = new Random(semilla * 31 + indice); // local: no hay
estado compartido
        String id = String.format("U%06d", indice);
        int viajes = 1 + rnd.nextInt(3);
        List<List<Integer>> franjas = switch (viajes) {
            case 1 -> List.of(MANANA);
            case 2 -> List.of(MANANA, TARDE);
            default -> List.of(MANANA, MEDIODIA, TARDE);
        };
        return franjas.stream().flatMap(franja -> viaje(id, franja, rnd));
    }

    private static Stream<Registro> viaje(String id, List<Integer> franja,
Random rnd) {
        String ruta =
RUTAS_PONDERADAS.get(rnd.nextInt(RUTAS_PONDERADAS.size()));
        List<String> estaciones = RUTAS.get(ruta);
        int origen = rnd.nextInt(estaciones.size());
        int destino = (origen + 1 + rnd.nextInt(estaciones.size() - 1)) %
estaciones.size();
        LocalDateTime inicio =
FECHA.atTime(franja.get(rnd.nextInt(franja.size())), rnd.nextInt(60),
rnd.nextInt(60));
        long minutos = Math.abs(destino - origen) * 6L + 2 + rnd.nextInt(10);
        return Stream.of(
                new Registro(id, ruta, estaciones.get(origen),
Accion.ENTRADA, inicio),
                new Registro(id, ruta, estaciones.get(destino),
Accion.SALIDA, inicio.plusMinutes(minutos)));
    }
}
```

AnalizadorTransporte:

```
package com.tecnomovil;

import com.tecnomovil.Registro.Accion;

import java.time.Duration;
import java.util.Collection;
import java.util.Collections;
import java.util.Comparator;
import java.util.List;
import java.util.Map;
import java.util.TreeMap;
import java.util.function.Function;
import java.util.function.Predicate;
import java.util.stream.Collectors;
import java.util.stream.IntStream;
import java.util.stream.Stream;

/*
Módulo de TecnoMovil
Con todas las operaciones siendo funciones puras:
Estas dependen solo de sus parámetros, no modifican la
lista recibida, ni usan estado compartido y devuelven
colecciones inmutables.
*/

public final class AnalizadorTransporte {
    private AnalizadorTransporte() {    }

/* Crea un stram scuencial o paralelo sin alterar la colección original */
```

```java
    public static final Predicate<Registro> ES_ENTRADA = r -> r.accion() ==
Accion.ENTRADA;
    public static final Predicate<Registro> TODOS = r -> true;
    public static final Function<Registro, Integer> HORA = r ->
r.timestamp().getHour();

    private static <T> Stream<T> flujo (Collection<T> datos, boolean
paralelo) {
        return paralelo ? datos.parallelStream() : datos.stream();
    }

/* Función de orden superior: esta recibe un filtro y una función de
clasificación y devuelve el conteo agrupado. */
public static <K extends Comparable<K>> Map<K, Long> contarPor(List<Registro>
registros, boolean paralelo,
            Predicate<Registro> filtro, Function<Registro, K> clave) {
        return flujo(registros, paralelo)
                .filter(filtro)
                .collect(Collectors.collectingAndThen(
                        Collectors.groupingBy(clave, TreeMap::new,
Collectors.counting()),
                        Collections::unmodifiableMap));
    }

    /* Devuelve n elementos con mayor conteo */
    public static <K> List<Map.Entry<K, Long>> topN(Map<K, Long> conteos, int
n) {
        return conteos.entrySet().stream()
                .sorted(Map.Entry.<K, Long>comparingByValue().reversed())
                .limit(n)
                .map(e -> Map.entry(e.getKey(), e.getValue()))
                .toList();
    }

    /* a) Afluencia según estación */
    /* Cantidad de usuarios que ingresan a cada estación */
    public static Map<String, Long> afluenciaPorEstacion(List<Registro>
registros, boolean paralelo) {
        return flujo(registros, paralelo)
                .filter(ES_ENTRADA)
                .collect(Collectors.collectingAndThen(
                        Collectors.groupingBy(Registro::estacion,
TreeMap::new,
                                Collectors.collectingAndThen(

Collectors.mapping(Registro::idUsuario, Collectors.toSet()),
                                        usuarios -> (long) usuarios.size())),
                        Collections::unmodifiableMap));
    }
    /* b) Horas pico */
    /* Número de registros por hora del día. */
    public static Map<Integer, Long> flujoPorHora(List<Registro> registros,
boolean paralelo) {
        return contarPor(registros, paralelo, TODOS, HORA);
    }

    public static List<Map.Entry<Integer, Long>> horasPico(List<Registro>
registros, int n, boolean paralelo) {
        return topN(flujoPorHora(registros, paralelo), n);
    }

    /* c) Rutas más usadas */
    public static Map<String, Long> usoPorRuta(List<Registro> registros,
boolean paralelo) {
```

```
        return contarPor(registros, paralelo, ES_ENTRADA, Registro::ruta);
    }

    public static List<Map.Entry<String, Long>>
rutasMasUtilizadas(List<Registro> registros, int n, boolean paralelo) {
        return topN(usoPorRuta(registros, paralelo), n);
    }

        /* d) Patrones de viaje por usuario */
        /* Para cada usuario, las estaciones en el orden en que aparecen en
los registros */
        public static Map<String, List<String>>
patronesPorUsuario(List<Registro> registros, boolean paralelo) {
            return flujo(registros, paralelo)
                    .collect(Collectors.collectingAndThen(
                            Collectors.groupingBy(Registro::idUsuario,
TreeMap::new,
                                    Collectors.mapping(Registro::estacion,
Collectors.toUnmodifiableList())),
                            Collections::unmodifiableMap));
        }

        /* e) Tiempo promedio entre estaciones */
        /* Desplazamiento de un usuario entre dos estaciones consecutivas */
        public record Tramo(String origen, String destino, long minutos) {
            public String clave() { return origen + " -> " + destino; }
        }

        /* Convierte los registros de un usuario en tramos (entrada seguida
de salida) */
        public static List<Tramo> tramos(List<Registro> registrosUsuario) {
            return IntStream.range(0, registrosUsuario.size() - 1)
                    .mapToObj(i -> Map.entry(registrosUsuario.get(i),
registrosUsuario.get(i + 1)))
                    .filter(par -> ES_ENTRADA.test(par.getKey()) &&
!ES_ENTRADA.test(par.getValue()))
                    .map(par -> new Tramo(par.getKey().estacion(),
par.getValue().estacion(),
                            Duration.between(par.getKey().timestamp(),
par.getValue().timestamp()).toMinutes()))
                    .toList();
        }

    /* Todos los tramos del día, usuario por usuario */
    private static Stream<Tramo> todosLosTramos(List<Registro> registros,
boolean paralelo) {
        Map<String, List<Registro>> porUsuario = flujo(registros, paralelo)
                .collect(Collectors.groupingBy(Registro::idUsuario));
        return flujo(porUsuario.values(), paralelo).flatMap(lista ->
tramos(lista).stream());
    }

    /* Todos los tramos del día, usuario por usuaior */
    public static Map<String, Double>
tiempoPromedioEntreEstaciones(List<Registro> registros, boolean paralelo) {
        return todosLosTramos(registros, paralelo)
                .collect(Collectors.collectingAndThen(
                        Collectors.groupingBy(Tramo::clave, TreeMap::new,
Collectors.averagingLong(Tramo::minutos)),
                        Collections::unmodifiableMap));
    }

    /* Tiempo promedio por cada par de estaciones de origen a destino */
```

```
    public static double tiempoPromedioGeneral(List<Registro> registros,
boolean paralelo) {
        return todosLosTramos(registros,
paralelo).mapToLong(Tramo::minutos).average().orElse(0);
    }

    /* f) Detección de sobrecarga */
    /* Estado de una ruta según ocupación */
    public record EstadoRuta(String ruta, int horaPico, long
pasajerosHoraPico, boolean critica) {
        public String etiqueta() { return critica ? "CRITICA" : "NORMAL"; }
    }

    /* Ocupación simulada = pasajeros usando rutas en horas pico */
    /* Si se supera el umbral, esta se marca como crítica */
    public static List<EstadoRuta> detectarSobrecarga(List<Registro>
registros, long umbral, boolean paralelo) {
        return flujo(registros, paralelo)
                .filter(ES_ENTRADA)
                .collect(Collectors.groupingBy(Registro::ruta,
Collectors.groupingBy(HORA, Collectors.counting())))
                .entrySet().stream()
                .map(ruta -> ruta.getValue().entrySet().stream()
                        .max(Map.Entry.<Integer,
Long>comparingByValue().thenComparing(Map.Entry.comparingByKey()))
                        .map(pico -> new EstadoRuta(ruta.getKey(),
pico.getKey(), pico.getValue(), pico.getValue() > umbral))
                        .orElseThrow())

.sorted(Comparator.comparingLong(EstadoRuta::pasajerosHoraPico).reversed())
                .toList();
    }

}
```

Main:

```
package com.tecnomovil;

import java.time.LocalDateTime;
import java.util.List;
import java.util.function.Supplier;


public class Main {

    private static final int USUARIOS = 200_000;
    private static final long SEMILLA = 42L;
    private static final long UMBRAL_PASAJEROS_HORA = 15_000;

    public static void main(String[] args) {
        long inicio = System.nanoTime();
        List<Registro> registros = GeneradorDatos.generar(USUARIOS, SEMILLA);
        titulo("TECNOMOVIL DATA - Procesamiento funcional de datos");
        System.out.printf("Registros generados: %,d de %,d usuarios (%d
ms)%n",
            registros.size(), USUARIOS, (System.nanoTime() - inicio) /
1_000_000);
        System.out.println("Muestra de registros:");
        registros.stream().limit(3).forEach(r -> System.out.println(" " +
r));

        titulo("a) Afluencia por estacion (usuarios distintos que
ingresan)");
```

```
        AnalizadorTransporte.afluenciaPorEstacion(registros, true)
                .forEach((estacion, n) -> System.out.printf("  %-18s
%,10d%n", estacion, n));

        titulo("b) Flujo por hora y horas pico");
        AnalizadorTransporte.flujoPorHora(registros, true)
                .forEach((hora, n) -> System.out.printf("  %02d:00  %,9d
%s%n", hora, n, "#".repeat((int) (n / 5_000))));
        System.out.println("  Top 3 horas pico:");
        AnalizadorTransporte.horasPico(registros, 3, true)
                .forEach(e -> System.out.printf("    %02d:00 -> %,d
registros%n", e.getKey(), e.getValue()));

        titulo("c) Rutas mas utilizadas (entradas)");
        AnalizadorTransporte.rutasMasUtilizadas(registros, 5, true)
                .forEach(e -> System.out.printf("  %-20s %,10d%n",
e.getKey(), e.getValue()));

        titulo("d) Patrones de viaje (primeros 5 usuarios)");
        AnalizadorTransporte.patronesPorUsuario(registros,
true).entrySet().stream()
                .limit(5)
                .forEach(e -> System.out.printf("  %s: %s%n", e.getKey(),
String.join(" > ", e.getValue())));

        titulo("e) Tiempo promedio entre estaciones (10 tramos mas largos)");
        AnalizadorTransporte.tiempoPromedioEntreEstaciones(registros,
true).entrySet().stream()
                .sorted(java.util.Map.Entry.<String,
Double>comparingByValue().reversed())
                .limit(10)
                .forEach(e -> System.out.printf("  %-36s %6.1f min%n",
e.getKey(), e.getValue()));
        System.out.printf("  Promedio general: %.1f min%n",
AnalizadorTransporte.tiempoPromedioGeneral(registros, true));

        titulo("f) Deteccion de sobrecarga (umbral " + UMBRAL_PASAJEROS_HORA
+ " pasajeros/hora)");
        AnalizadorTransporte.detectarSobrecarga(registros,
UMBRAL_PASAJEROS_HORA, true)
                .forEach(e -> System.out.printf("  %-20s pico %02d:00  %,8d
pasajeros  [%s]%n",
                        e.ruta(), e.horaPico(), e.pasajerosHoraPico(),
e.etiqueta()));

        titulo("Prueba de inmutabilidad");
        try {
            registros.add(new Registro("X", "X", "X",
Registro.Accion.ENTRADA, LocalDateTime.now()));
            System.out.println("  ERROR: la lista fue modificada");
        } catch (UnsupportedOperationException e) {
            System.out.println("  OK: la lista original no se puede modificar
(UnsupportedOperationException)");
        }

        titulo("Secuencial vs paralelo (todas las operaciones)");
        ejecutarTodo(registros, false); // calentamiento de la JVM antes de
medir
        ejecutarTodo(registros, true);
        Object secuencial = medir("stream()        ", () ->
ejecutarTodo(registros, false));
        Object paralelo = medir("parallelStream()", () ->
ejecutarTodo(registros, true));
```

```
        System.out.println("  Resultados identicos: " +
secuencial.equals(paralelo)
                + " (sin condiciones de carrera)");
    }

    private static List<Object> ejecutarTodo(List<Registro> r, boolean p) {
        return List.of(
                AnalizadorTransporte.afluenciaPorEstacion(r, p),
                AnalizadorTransporte.flujoPorHora(r, p),
                AnalizadorTransporte.usoPorRuta(r, p),
                AnalizadorTransporte.patronesPorUsuario(r, p),
                AnalizadorTransporte.tiempoPromedioEntreEstaciones(r, p),
                AnalizadorTransporte.detectarSobrecarga(r,
UMBRAL_PASAJEROS_HORA, p));
    }

    private static <T> T medir(String etiqueta, Supplier<T> tarea) {
        long t = System.nanoTime();
        T resultado = tarea.get();
        System.out.printf("  %s %,6d ms%n", etiqueta, (System.nanoTime() - t)
/ 1_000_000);
        return resultado;
    }

    private static void titulo(String texto) {
        System.out.println();
        System.out.println("=== " + texto + " ===");

    }

}
```

Conceptos funcionales

Funciones puras:

Este tipo de función depende solo de sus parámetros, devolviendo siempre el mismo resultado ante la misma entrada y sin producir efectos secundarios, con esto en cuenta, todos los métodos de AnalizadorTransporte cumplen esta condición; recibiendo la lista de registros y dando un resultado nuevo sin modificarla.

Inmutabilidad

Esta característica garantiza que los datos no cambien una vez fueron creados, **Registro** fue definida como **record**, en base a esto sus campos son **final**. El generador entrega entonces la lista con **toList()**, que no admite modificaciones y con resultados que se dan con **Collections::unmodifiableMap** o **toUnmodifiableList().**

Expresiones lambda

Estas son funciones anónimas que se tratan como valores, siendo definidas como constantes reutilizables:

- ES_ENTRADA = r -> r.accion() == Accion.ENTRADA
- TODOS = r -> true
- HORA r -> r.timestamp() .getHour()

Streams

Estos permiten procesar colecciones de forma declarativa por medio de pipelines de operaciones, tales como **filter, map, flatMal, groupingBy, sorted** y **collect.** Con el método

flujo se permite elegir entre **stream()** y **parallelStream()** sin tener que cambiar el resto del código.

Evidencia de ejecución

a) Afluencia por estación

```
=== a) Afluencia por estacion (usuarios distintos que ingresan) ===
  Banderas                21,392
  Calle 100               25,374
  Calle 6                 24,005
  Calle 72                40,266
  Centro                  47,397
  La Paz                  10,906
  Las Aguas               26,538
  Museo                   40,222
  Portal Norte            25,609
  Portal Occidente        21,467
  Portal Sur              23,996
  Ricaurte                43,567
  San Cristobal           10,936
  Universidades           16,177
```

b) Flujo por horas y horas pico

```
=== b) Flujo por hora y horas pico ===
  05:00     38,342  #######
  06:00     82,595  ################
  07:00    127,191  ########################
  08:00     95,128  ##################
  09:00     50,556  ##########
  10:00      6,188  #
  11:00     22,918  ####
  12:00     49,632  #########
  13:00     30,369  ######
  14:00     26,711  #####
  15:00      3,680
  16:00     28,735  #####
  17:00     62,023  ############
  18:00     95,314  ##################
  19:00     42,513  ########
  20:00     33,499  ######
  21:00      4,584
  Top 3 horas pico:
    07:00 -> 127,191 registros
    18:00 -> 95,314 registros
    08:00 -> 95,128 registros
```

c) Rutas más utilizadas

```
=== c) Rutas mas utilizadas (entradas) ===
  T1-Troncal Norte        133,315
  T2-Troncal Sur           99,996
  A1-Alim. Occidente       66,711
  T3-Av. Oriental          66,634
  A2-Alim. Oriente         33,333
```

d) Patrones de viaje

```
=== d) Patrones de viaje (primeros 5 usuarios) ===
  U000000: Centro > Calle 6
  U000001: Portal Norte > Calle 72 > Calle 100 > Centro
  U000002: Portal Norte > Calle 72
  U000003: Universidades > Museo > Ricaurte > Portal Occidente
  U000004: Portal Sur > Centro > Centro > Calle 6 > Ricaurte > Centro
```

e) Tiempo promedio entre estaciones

```
=== e) Tiempo promedio entre estaciones (10 tramos mas largos) ===
  Museo -> Portal Norte                  30.5 min
  Portal Norte -> Museo                  30.5 min
  Museo -> Calle 100                     24.5 min
  Portal Sur -> Centro                   24.5 min
  Calle 100 -> Museo                     24.5 min
  Centro -> Portal Sur                   24.5 min
  Centro -> Portal Norte                 24.5 min
  Portal Norte -> Centro                 24.5 min
  Calle 72 -> Museo                      21.2 min
  Museo -> Calle 72                      21.2 min
  Promedio general: 16.7 min
```

f) Detección de sobrecarga

```
=== f) Deteccion de sobrecarga (umbral 15000 pasajeros/hora) ===
  T1-Troncal Norte     pico 07:00    22,220 pasajeros  [CRITICA]
  T2-Troncal Sur       pico 07:00    16,684 pasajeros  [CRITICA]
  A1-Alim. Occidente   pico 07:00    11,126 pasajeros  [NORMAL]
  T3-Av. Oriental      pico 07:00    11,098 pasajeros  [NORMAL]
  A2-Alim. Oriente     pico 07:00     5,545 pasajeros  [NORMAL]
```

Conclusiones

Esta aplicación por medio de programación funcional permitió dar con un módulo de procesamiento de datos claro, modular y fácil de mantener, con el uso de funciones puras y

colecciones inmutables se lograron eliminar efectos secundarios. Por el otro lado, las lambdas, las funciones de orden superior y los Streams dieron con la reducción del código necesario para la pertinente solución de los seis análisis requeridos.

Con entonces un enfoque funcional fue posible paralelizar el procesamiento de forma segura, con aproximadamente 8000.000 registros, se logró reducir el tiempo de ejecución a cerca de la mitad produciendo así resultados similares al del modo secuencial.